

Thema: Einstieg in die Interrupt Behandlung

Letzter Abgabetermin: Im letzten Praktikumstermin

Aufgabe Modifizieren Sie das Programm zum Auswerten des Drehgebers aus Aufgabe 2 so, dass die Erfassung und Auswertung der Signale in Interrupt-Service-Routinen erfolgt. Damit ist auch bei höherer Drehgeschwindigkeit eine zuverlässige Erkennung der Drehbewegung möglich. Die beiden Signalleitungen des Drehgebers werden an die Buchsen AUX0 und AUX1 des ITS-Boards angeschlossen.

Die maximale Drehgeschwindigkeit, die Ihr Programm verarbeiten kann, hängt nun insbesondere von den maximalen Ausführungszeiten der ISRs ab. Reduzieren Sie die maximalen Ausführungszeiten der ISRs so weit wie möglich. Insbesondere sollten Funktionen, die in der ISR aufgerufen werden, **static inline** sein. Damit spart man sich die Parameterübergabe über den Stack sowie den Funktionseinsprung.

Diskussion des Zeitverhaltens

- Messen Sie mit einem Oszilloskop die Ausführungszeiten der ISRs aus – sie schwanken vermutlich leicht.
- Erhöhen Sie das Optimierungslevel des Compilers, indem Sie im CMSIS View über „Manage Solution Settings“ (das Einstellungsrad) den „Build Type“ auf Release stellen und in der Datei *.csolution.yml für Release den Parameter optimize auf speed setzen. Messen Sie erneut die Laufzeit der ISR und vergleichen Sie diese mit der vorherigen Messung.
- Messen Sie bitte die maximale Zeit von der Änderung eines Signals des Drehgebers bis zum Einlesen seines Werts in der ISR.
- Bestimmen Sie die maximale Winkelgeschwindigkeit, die Ihr Programm verarbeiten kann.
- Diskutieren Sie die Abweichung der Winkelgeschwindigkeit bei hohen Drehgeschwindigkeiten (sollte kleiner als 20 ppm sein).

Hinweise

- Zum fehlerfreien Auslesen des Drehgebers müssen alle Phasenwechsel des Drehgebers erkannt werden. Im Gegensatz zur Aufgabe 2 definiert nicht mehr die maximale Ausführungszeit eines Super-Loop-Durchlaufs sondern die maximale Ausführungszeit einer ISR die maximale Drehgeschwindigkeit, die das Programm verarbeiten kann. Zur flüssigen Aktualisierung des Displays wird auch noch ein wenig Rechenzeit zwischen zwei ISR-Aufrufen benötigt. Somit sollten in der ISR möglichst wenig Berechnungen durchgeführt werden. Typischerweise werden der aktuelle Zeitstempel, die Überprüfung des Phasenwechsels mit Fehlererkennung und Drehrichtungsbestimmung sowie der Phasenwechselzähler in der ISR berechnet.
- Die an AUX0 und AUX1 gebundenen Pins sowie deren Ports sind nicht initialisiert.
- Da ausschließlich ein Flankenwechsel auf AUX0 bzw. AUX1 die Ausführung der entsprechenden ISR anstößt, müssen auch hier in der Initialisierung die aktuelle Phase -also die Werte von AUX0 und AUX1 - eingelesen werden.
- Je näher der Zeitstempel eines Phasenwechsels am Zeitpunkt des Phasenwechsels – also am Zeitpunkt der steigenden oder fallenden Flanke liegt, desto genauer ist die Berechnung der Winkelgeschwindigkeit. Üblicherweise wird ein Zeitstempel, der zeitgleich zu einem Flankenwechsel genommen werden soll, mit dem Input Capture Mode des Timers erfasst.

Hier sparen wir uns den Einarbeitungsaufwand und lesen stattdessen den Zeitstempel so früh wie möglich in der ISR.

- Die ISR aktualisiert die Variablen des Zeitstempels und des Phasenwechselzählers. Der Zeitstempel muss den Zeitpunkt der letzten Aktualisierung des Phasenwechselzählers enthalten. Werden die Lesezugriffe auf die beiden Variablen durch eine ISR unterbrochen, beziehen sich die Werte der beiden Variablen auf unterschiedliche Phasenwechsel und die Berechnung der Winkelgeschwindigkeit ist falsch. Die ISR selbst wird nicht unterbrochen (analog zur Diskussion der read-modify-write Zugriffe in der VL). Ihre Software stellt wie folgt sicher, dass Zeitstempel und Phasenwechselzähler zum selben Phasenwechsel gehören: Zuerst wird der Zeitstempel, dann der Phasenwechselzähler gelesen. Anschließend werden beide Werte nochmals gelesen. Haben Sie beim zweiten Lesen Phasenwechselzähler und Zeitstempel nicht verändert, dann wurde die Lesephase nicht durch die ISR unterbrochen. Ansonsten wird das Verfahren wiederholt. Wichtig ist, dass die Zustandsvariablen `volatile` sind. Nach zehn fehlerhaften Versuchen geht das Programm in den Fehlerzustand über – scheinbar ist dann die Drehgeschwindigkeit zu hoch.
- Der Timer, der die Zeitstempel zur Berechnung von Zeitfenstern liefert, basiert auf der Frequenz eines Oszillators. Aus unterschiedlichen Gründen schwingt ein Oszillator nicht ganz genau, sondern weicht von der „realen Zeit“, wie zum Beispiel der Zeit einer Atomuhr, ab. Dies kann man mit einer Uhr vergleichen, die etwas zu schnell oder zu langsam läuft. Ebenso basiert die Erzeugung der Rechtecksignale auf dem Pi Pico auf einem Timer. Somit laufen die Uhren des Rechteckgenerators auf dem Pi Pico und dem ITS-Board unterschiedlich schnell. Die Abweichung der Oszillatoren wird im ppm (part per million) angegeben. Dies hat Auswirkungen auf die Genauigkeit der Berechnung der Winkelgeschwindigkeit. Bei kleinen Winkelgeschwindigkeiten ist der resultierende Fehler so klein, dass er bei der Anzeige nicht auffällt. Bei hohen Winkelgeschwindigkeiten fällt er auf und sollte sich proportional zur Winkelgeschwindigkeit vergrößern.

Folgende Unterlagen müssen am Abend vor dem letzten Praktikumstermin im git Repo auf dem Server hochgeladen und mit einem aussagekräftigen Bezeichner „getagged“ sein:

- Darstellung der Änderungen gegenüber Aufgabe 2.
- Ansatz zur Bestimmung der maximalen Drehgeschwindigkeit, die Ihr Programm verarbeiten kann.

Abnahme der Aufgabe im Praktikum

- Das Konzept muss vorliegen.
- Kommentierter Quellcode, der dem C-Coding Style entspricht, muss vorliegen.
- Sie müssen Ihr Programm erklären können.
- Der Drehwinkel wird exakt ausgegeben. Bei hohen Geschwindigkeiten sollte die gemessene Winkelgeschwindigkeit maximal 1% von der generierten Winkelgeschwindigkeit abweichen.
- Eine Liste der von Ihnen durchgeführten Tests - zusammen mit den Testergebnissen - muss vorliegen. Überlegen Sie sich sinnvolle Testfälle.
- Die oben geforderte Analyse des Zeitverhaltens muss vorliegen.